МОДУЛЬ 3

# Кластеризация в Python

Разработчик интеллектуальных решений
Базовый уровень для студентов СПО

# Главная идея

01 Кластеризация ищет группы похожих объектов по признакам.

02 Сходство определяется по значениям выбранных признаков.

03 KMeans ищет центры заданного количества кластеров и относит каждый объект к ближайшему центру.

04 Важно различать результат алгоритма и его объяснение: KMeans создаёт группы по числам, а смысловые названия вроде «быстрые» или «защитники» появляются только после анализа признаков.

# Кейс урока · таблица 1/2

| character | speed | power | defense |
|---|---|---|---|
| Rys | 9 | 3 | 4 |
| Grom | 2 | 9 | 8 |

# Кейс урока · таблица 2/2

| character | speed | power | defense |
|---|---|---|---|
| Klinok | 6 | 7 | 5 |
| Medik | 4 | 4 | 8 |

# Ключевые понятия

01 объект - отдельная строка таблицы, например один персонаж.

02 признак - числовая характеристика объекта, например speed или power.

03 кластер - группа объектов, похожих по признакам.

04 центр кластера - точка, относительно которой KMeans собирает близкие объекты.

# Кластеризация и классификация

01 В классификации модель обучается на правильных ответах: есть признаки X и известные классы y.

02 Например, в задаче классификации для каждого персонажа мог бы быть заранее указан класс «разведчик», «боец» или «защитник».

03 Поэтому кластер нельзя автоматически считать настоящим классом объекта.

# Как выглядит работа в Python

```
import pandas as pd
from sklearn.cluster import KMeans
df = pd.read_csv('game_characters_sklearn.csv')
X = df[['speed', 'power']]
model = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster'] = model.fit_predict(X)
df.head()
```

Сначала необходимо загрузить таблицу и отделить признаки, по которым алгоритм будет сравнивать персонажей.

# Визуализация кластеров

```
import seaborn as sns
import matplotlib.pyplot as plt
sns.scatterplot(
data=df,
x='speed',
y='power',
hue='cluster'
```

Фрагмент примера. Полный код приведён в тексте лекции.

# Интерпретация результата

```
df.groupby('cluster')[['speed', 'power', 'defense']].mean().round(1)
```

Одного цвета точек недостаточно, чтобы понять содержание группы.

Команда groupby('cluster') собирает строки с одинаковым номером кластера, mean() вычисляет среднее значение каждого выбранного признака, а round(1) округляет результат до одного знака после запятой.

# Почему результат может меняться

01 Количество кластеров KMeans не определяет самостоятельно: его задают до запуска модели с помощью n_clusters.

02 Если изменить число кластеров, алгоритм разделит объекты иначе.

03 Результат также зависит от выбранных признаков.

04 Кроме того, сами номера групп могут не нести постоянного смысла: группа быстрых персонажей в одном запуске может получить номер 0, а в другом условии — номер 2.

# Итог

01 К концу урока обучающиеся загружают датасет, выбирают признаки, запускают KMeans, получают номера кластеров, строят карту групп и объясняют смысл найденных кластеров через признаки объектов.

02 Полный ход работы можно представить как последовательность: выбрать числовые признаки X, задать количество кластеров, выполнить fit_predict(), добавить метки в таблицу, построить диаграмму и сравнить средние характеристики групп.

03 Ключевой вывод: алгоритм выполняет числовое разбиение объектов, но содержательная интерпретация результата остаётся задачей исследователя.

# Практическая работа

01 Задание №1

02 Задание №2

03 Задание №3

# Проверка результата

01 Сопоставление результата с условием

02 Проверка на исходных и изменённых данных

03 Объяснение решения своими словами